

EN3160 – Computer Vision

Assignment 01: Intensity Transformations and Neighborhood Filtering

Name: Saruka U.

Index Number: 230585C

Question 1 – Intensity Transformation

```
In [3]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

breakpoints = np.array([
    [0, 0],
    [50, 50],
    [100, 150],
    [150, 255],
    [255, 255]
])

def intensity_transform(im, breakpoints):
    # Make sure the breakpoints are sorted by input intensity
    breakpoints = np.asarray(breakpoints, dtype=np.float32)
    breakpoints = breakpoints[np.argsort(breakpoints[:, 0])]

    # Create output image with floating-point values
    output = np.zeros_like(im, dtype=np.float32)

    # Process each interval between consecutive breakpoints
    for i in range(len(breakpoints) - 1):

        x1, y1 = breakpoints[i]
        x2, y2 = breakpoints[i + 1]

        # Pixels belonging to this intensity interval
        mask = (im >= x1) & (im <= x2)

        # Piecewise linear interpolation
        output[mask] = (
            y1 + (y2 - y1) / (x2 - x1) * (im[mask] - x1)
        )

    # Handle values below the first breakpoint
    output[im < breakpoints[0, 0]] = breakpoints[0, 1]

    # Handle values above the last breakpoint
    output[im > breakpoints[-1, 0]] = breakpoints[-1, 1]

    # Convert back to 8-bit image
```

```

output = np.clip(output, 0, 255).astype(np.uint8)

return output

```

```

In [4]: x = np.arange(256, dtype=np.uint8)

# Treat the intensity values as a 1 x 256 image
x_image = x.reshape(1, -1)

# Apply the transformation
y = intensity_transform(x_image, breakpoints)[0]

plt.figure(figsize=(7, 5))

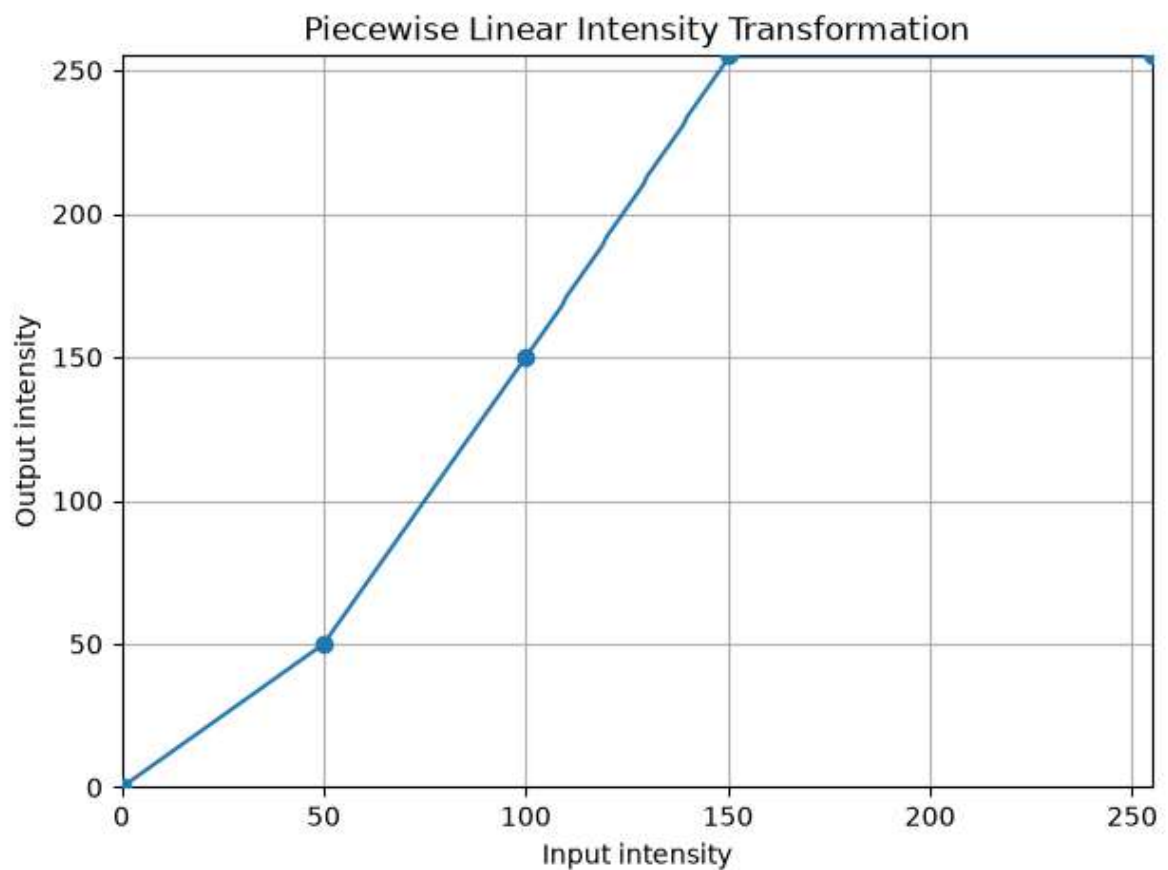
plt.plot(x, y)

# Show the specified breakpoints
plt.scatter(
    breakpoints[:, 0],
    breakpoints[:, 1],
    zorder=3
)

plt.xlabel("Input intensity")
plt.ylabel("Output intensity")
plt.title("Piecewise Linear Intensity Transformation")
plt.xlim(0, 255)
plt.ylim(0, 255)
plt.grid(True)

plt.show()

```



```
In [5]: im = cv.imread("images/q1/fig1.jpg", cv.IMREAD_GRAYSCALE)

transformed = intensity_transform(im, breakpoints)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))

ax[0].imshow(im, cmap="gray", vmin=0, vmax=255)
ax[0].set_title("Original Image")
ax[0].axis("off")

ax[1].imshow(transformed, cmap="gray", vmin=0, vmax=255)
ax[1].set_title("Transformed Image")
ax[1].axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Transformed Image



```
In [6]: breakpoints_1 = np.array([
    [0, 0],
    [50, 60],
    [100, 130],
    [150, 140],
    [255, 240]
])

breakpoints_2 = np.array([
    [0, 0],
    [60, 45],
    [120, 115],
    [150, 190],
    [255, 255]
])

breakpoints_3 = np.array([
    [0, 5],
    [60, 50],
    [120, 135],
    [180, 210],
    [255, 250]
])
```

```

transformed_1 = intensity_transform(im, breakpoints_1)
transformed_2 = intensity_transform(im, breakpoints_2)
transformed_3 = intensity_transform(im, breakpoints_3)

fig, ax = plt.subplots(1, 4, figsize=(16, 5))

ax[0].imshow(im, cmap="gray", vmin=0, vmax=255)
ax[0].set_title("Original")
ax[0].axis("off")

ax[1].imshow(transformed_1, cmap="gray", vmin=0, vmax=255)
ax[1].set_title("Experiment 1")
ax[1].axis("off")

ax[2].imshow(transformed_2, cmap="gray", vmin=0, vmax=255)
ax[2].set_title("Experiment 2")
ax[2].axis("off")

ax[3].imshow(transformed_3, cmap="gray", vmin=0, vmax=255)
ax[3].set_title("Experiment 3")
ax[3].axis("off")

plt.tight_layout()
plt.show()

```



```

In [7]: x = np.arange(256, dtype=np.uint8)
x_image = x.reshape(1, -1)

# Apply the transformation to all intensity values
y = intensity_transform(x_image, breakpoints_3)[0]

plt.figure(figsize=(7, 5))

plt.plot(x, y, linewidth=2, label="Experiment 3 Transformation")
plt.scatter(
    breakpoints_3[:, 0],
    breakpoints_3[:, 1],
    color="red",
    s=50,
    zorder=3,
    label="Breakpoints"
)

plt.title("Experiment 3: Intensity Transformation")
plt.xlabel("Input intensity")
plt.ylabel("Output intensity")

plt.xlim(0, 255)
plt.ylim(0, 255)

```

```
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend()

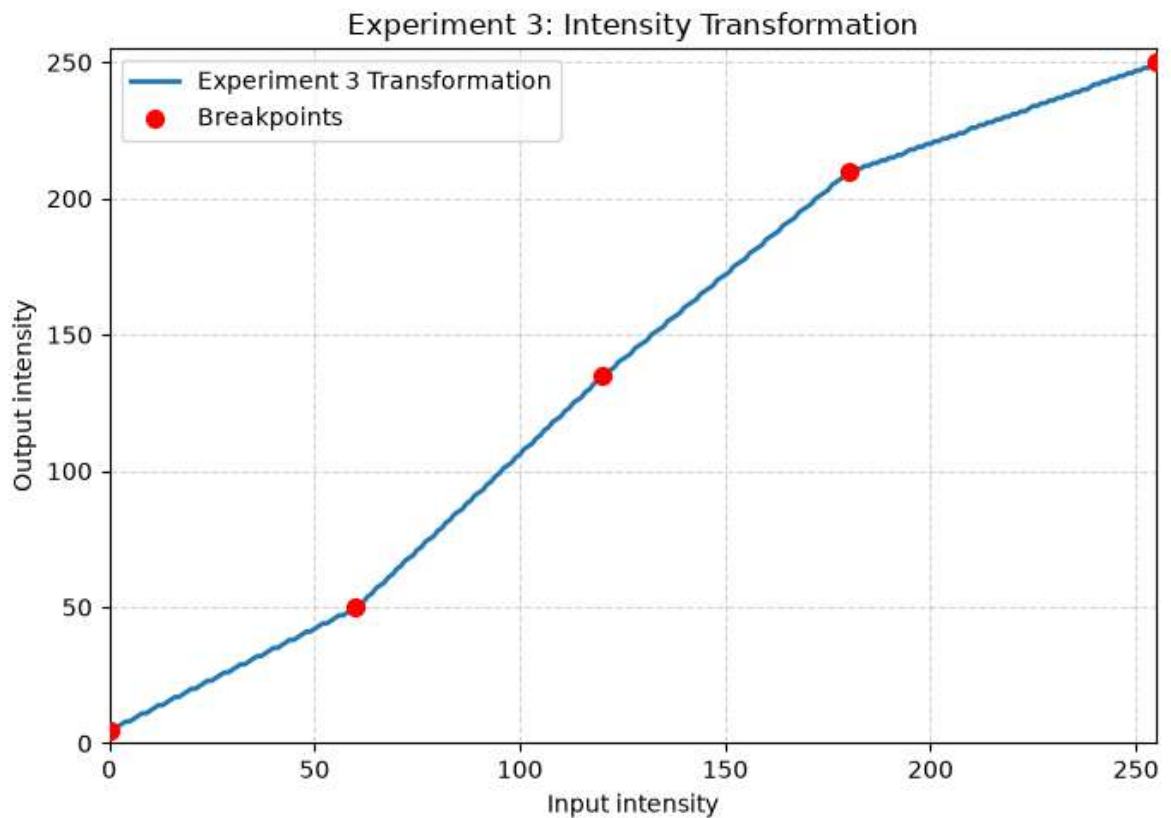
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(1, 2, figsize=(12, 5))

ax[0].imshow(im, cmap="gray", vmin=0, vmax=255)
ax[0].set_title("Original Image")
ax[0].axis("off")

ax[1].imshow(transformed_3, cmap="gray", vmin=0, vmax=255)
ax[1].set_title("Transformed Image (Experiment 3)")
ax[1].axis("off")

plt.tight_layout()
plt.show()
```





Selection of the Best Transformation

Three different sets of breakpoints were tested to obtain a visually pleasing intensity transformation. Experiment 1 provides a relatively mild enhancement, while Experiment 2 produces stronger contrast and brightness but causes some bright regions of the face to become over-enhanced. Experiment 3 provides a better balance between contrast enhancement and preservation of image details. Therefore, Experiment 3 was selected as the most visually pleasing transformation.

Question 2 — Intensity Transformation for Brain Tissue

```
In [8]: # Load the brain proton density image
brain = cv.imread("images/q2/fig2.png", cv.IMREAD_GRAYSCALE)

print("Image shape:", brain.shape)
print("Image dtype:", brain.dtype)

plt.figure(figsize=(6, 6))
plt.imshow(brain, cmap="gray", vmin=0, vmax=255)
plt.title("Brain Proton Density Image")
plt.axis("off")
plt.show()
```

Image shape: (588, 495)
Image dtype: uint8

Brain Proton Density Image



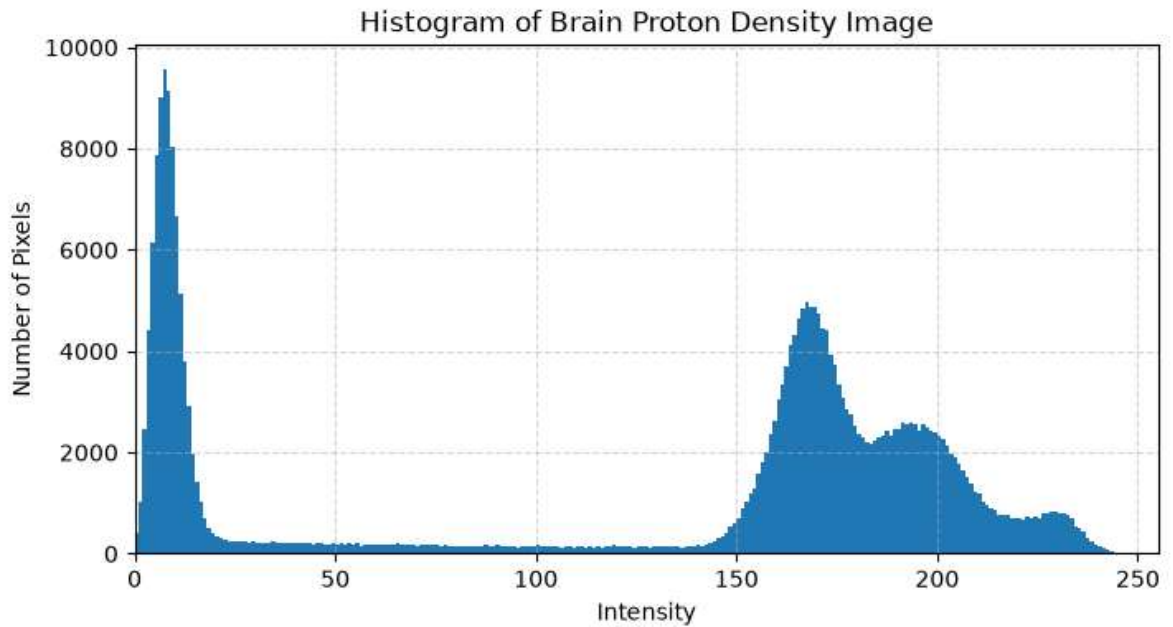
```
In [9]: plt.figure(figsize=(8, 4))

plt.hist(brain.ravel(), bins=256, range=(0, 256))

plt.title("Histogram of Brain Proton Density Image")
plt.xlabel("Intensity")
plt.ylabel("Number of Pixels")

plt.xlim(0, 255)
plt.grid(True, linestyle="--", alpha=0.5)

plt.show()
```



```
In [10]: breakpoints_wm = np.array([
    [0, 0],
    [160, 50],
    [180, 150],
    [220, 230],
    [255, 255]
])

breakpoints_gm = np.array([
    [0, 0],
    [140, 40],
    [155, 120],
    [180, 240],
    [255, 255]
])

# Create intensity values from 0 to 255
x = np.arange(256, dtype=np.uint8)
x_image = x.reshape(1, -1)

# Calculate transformation curves
y_wm = intensity_transform(x_image, breakpoints_wm)[0]
y_gm = intensity_transform(x_image, breakpoints_gm)[0]

# Plot transformations
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

ax[0].plot(x, y_wm)
ax[0].scatter(
    breakpoints_wm[:, 0],
    breakpoints_wm[:, 1],
    color="red",
    zorder=3
)
ax[0].set_title("White Matter Intensity Transformation")
ax[0].set_xlabel("Input intensity")
ax[0].set_ylabel("Output intensity")
ax[0].set_xlim(0, 255)
ax[0].set_ylim(0, 255)
```

```

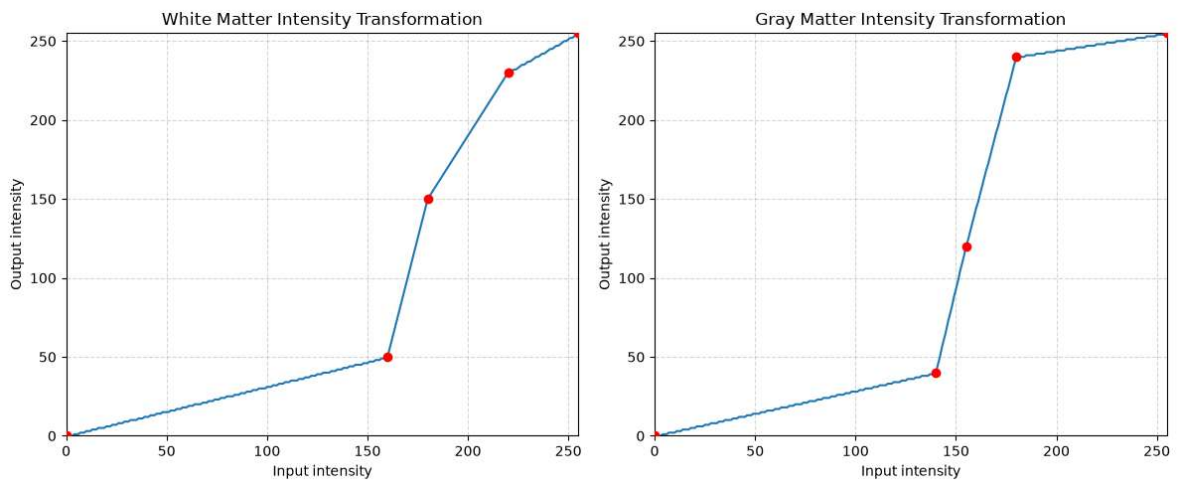
ax[0].grid(True, linestyle="--", alpha=0.5)

ax[1].plot(x, y_gm)
ax[1].scatter(
    breakpoints_gm[:, 0],
    breakpoints_gm[:, 1],
    color="red",
    zorder=3
)

ax[1].set_title("Gray Matter Intensity Transformation")
ax[1].set_xlabel("Input intensity")
ax[1].set_ylabel("Output intensity")
ax[1].set_xlim(0, 255)
ax[1].set_ylim(0, 255)
ax[1].grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()

```



```

In [11]: # Apply the transformations
brain_wm = intensity_transform(brain, breakpoints_wm)
brain_gm = intensity_transform(brain, breakpoints_gm)

# Display results
fig, ax = plt.subplots(1, 3, figsize=(15, 5))

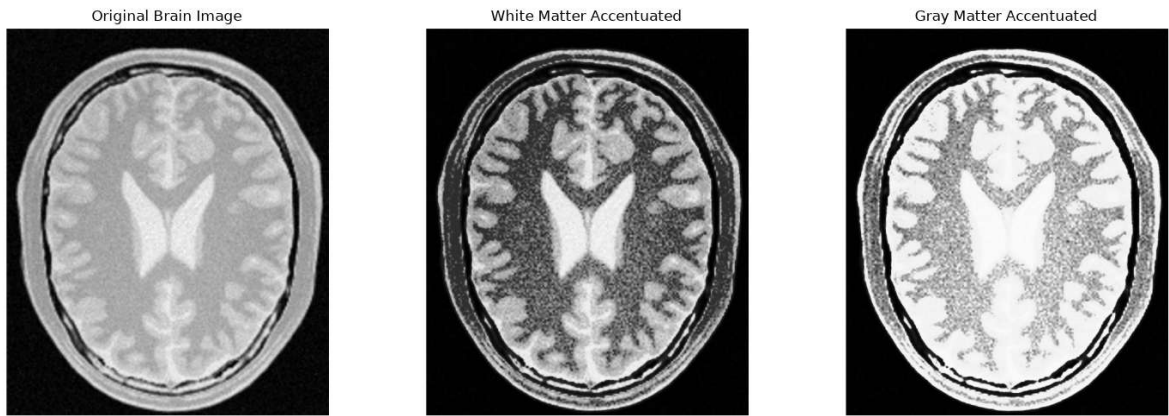
ax[0].imshow(brain, cmap="gray", vmin=0, vmax=255)
ax[0].set_title("Original Brain Image")
ax[0].axis("off")

ax[1].imshow(brain_wm, cmap="gray", vmin=0, vmax=255)
ax[1].set_title("White Matter Accentuated")
ax[1].axis("off")

ax[2].imshow(brain_gm, cmap="gray", vmin=0, vmax=255)
ax[2].set_title("Gray Matter Accentuated")
ax[2].axis("off")

plt.tight_layout()
plt.show()

```



Breakpoint Selection

The breakpoint values were selected experimentally based on the intensity distribution of the brain proton-density image. The transformations were adjusted to increase the contrast of the intensity regions associated with the target tissue while keeping the background relatively dark. The final breakpoint sets were selected based on the visual separation and visibility of the respective tissue regions.

Question 3 — Gamma Correction

Gamma correction is applied to the lightness (L) plane of the image in the *Lab** colour space. The L plane represents the lightness information, while the a and b planes contain colour information.

The image is converted from RGB to *Lab**, gamma correction is applied only to the L plane, and the modified L plane is then recombined with the original a and b planes. The result is converted back to RGB.

The gamma value is selected experimentally to obtain a visually pleasing result.

```
In [12]: im_q3 = cv.imread("images/q3/fig3.png")

# Convert BGR to RGB for displaying with matplotlib
im_q3_rgb = cv.cvtColor(im_q3, cv.COLOR_BGR2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(im_q3_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Original Image



```
In [13]: # Convert the image from BGR to Lab colour space
lab = cv.cvtColor(im_q3, cv.COLOR_BGR2LAB)

# Split into L, a and b planes
L, a, b = cv.split(lab)

print("L range:", L.min(), "to", L.max())
print("a range:", a.min(), "to", a.max())
print("b range:", b.min(), "to", b.max())
```

```
L range: 0 to 255
a range: 122 to 160
b range: 125 to 170
```

```
In [14]: def apply_gamma_correction(L, gamma):
# Normalize L plane to [0, 1]
L_normalized = L.astype(np.float32) / 255.0

# Apply gamma correction
L_corrected = np.power(L_normalized, gamma)

# Convert back to [0, 255]
L_corrected = np.clip(L_corrected * 255, 0, 255).astype(np.uint8)

# Combine corrected L with original a and b planes

lab_corrected = cv.merge([
    L_corrected,
    a,
    b
])

# Convert Lab back to BGR
corrected_bgr = cv.cvtColor(lab_corrected, cv.COLOR_LAB2BGR)
```

```

# Convert BGR to RGB for matplotlib
corrected_rgb = cv.cvtColor(corrected_bgr, cv.COLOR_BGR2RGB)
return corrected_rgb

fig, ax = plt.subplots(2, 2, figsize=(15, 10))

ax[0,0].imshow(im_q3_rgb)
ax[0,0].set_title("Original Image")
ax[0,0].axis("off")

ax[1,0].imshow(apply_gamma_correction(L,gamma=0.3))
ax[1,0].set_title(f"Gamma Corrected Image (r = 0.3)")
ax[1,0].axis("off")

ax[0,1].imshow(apply_gamma_correction(L,gamma=0.9))
ax[0,1].set_title(f"Gamma Corrected Image (r = 0.9)")
ax[0,1].axis("off")

ax[1,1].imshow(apply_gamma_correction(L,gamma=1.6))
ax[1,1].set_title(f"Gamma Corrected Image (r = 1.6)")
ax[1,1].axis("off")

plt.tight_layout()
plt.show()

```



```

In [15]: def gamma_correct_L(L, gamma):
          L_normalized = L.astype(np.float32) / 255.0
          L_corrected = np.power(L_normalized, gamma)
          return np.clip(L_corrected * 255, 0, 255).astype(np.uint8)

          L_gamma03 = gamma_correct_L(L, 0.3)
          L_gamma09 = gamma_correct_L(L, 0.9)
          L_gamma16 = gamma_correct_L(L, 1.6)

          plt.figure(figsize=(9, 5))

```

```
fig, ax = plt.subplots(2, 2, figsize=(15, 10))

ax[0,0].hist(
    L.ravel(),
    bins=256,
    range=(0, 256),
    alpha=0.5,
    label="Original L plane"
)

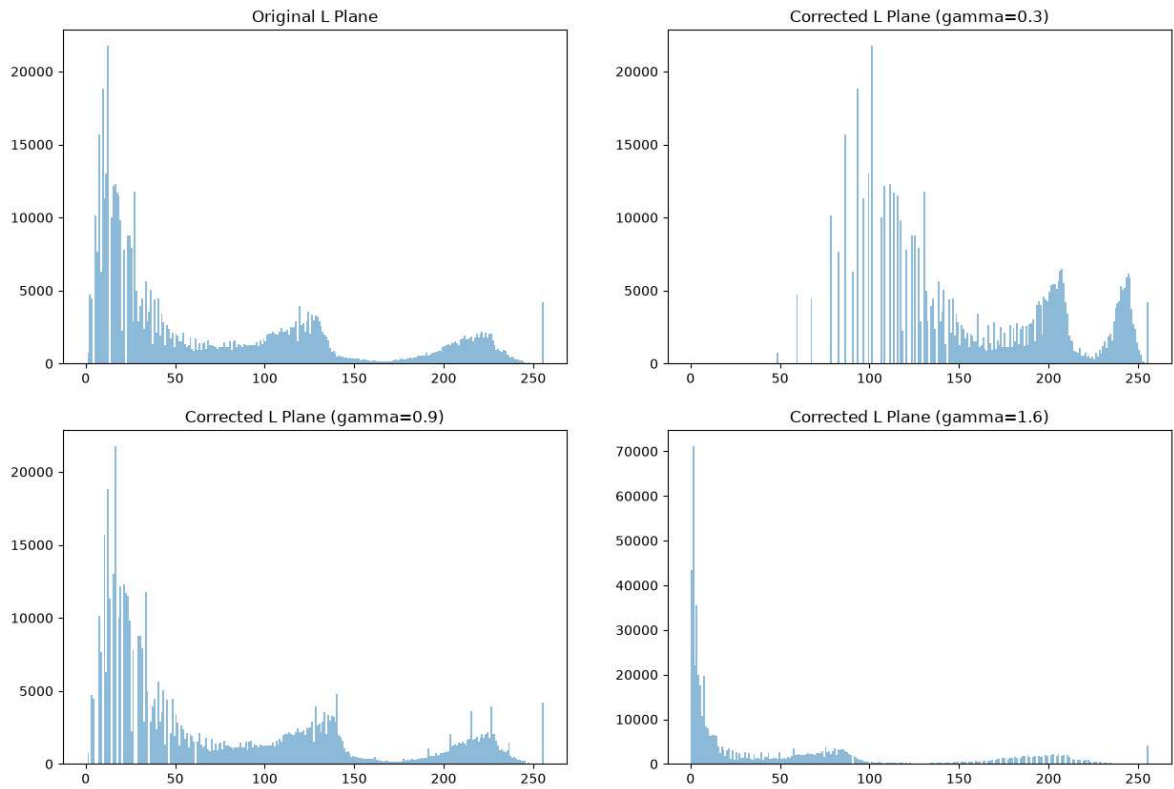
ax[0,1].hist(
    L_gamma03.ravel(),
    bins=256,
    range=(0, 256),
    alpha=0.5,
    label="Corrected L plane (gamma=0.3)"
)

ax[1,0].hist(
    L_gamma09.ravel(),
    bins=256,
    range=(0, 256),
    alpha=0.5,
    label="Corrected L plane (gamma=0.9)"
)

ax[1,1].hist(
    L_gamma16.ravel(),
    bins=256,
    range=(0, 256),
    alpha=0.5,
    label="Corrected L plane (gamma=1.6)"
)

ax[0,0].set_title("Original L Plane")
ax[0,1].set_title("Corrected L Plane (gamma=0.3)")
ax[1,0].set_title("Corrected L Plane (gamma=0.9)")
ax[1,1].set_title("Corrected L Plane (gamma=1.6)")
plt.show()
```

<Figure size 900x500 with 0 Axes>



Observation

Applying gamma correction only to the L plane changes the lightness of the image while preserving the colour information represented by the a and b^* planes.

Question 4 — Vibrance Enhancement

Vibrance enhancement is performed by applying an intensity transformation to the saturation plane of an image in the HSV colour space. The transformation increases saturation primarily around intermediate saturation values while limiting the output to the valid range of 0 to 255.

The transformation is given by

$$f(x) = \min \left(x + a(128)e^{-\frac{(x-128)^2}{2\sigma^2}}, 255 \right)$$

where (x) is the input saturation intensity, $a \in [0, 1]$ controls the strength of the enhancement, and $\sigma = 70$.

(a) Convert the image to HSV colour space

```
In [16]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

# Load image
im_q4 = cv.imread("images/q4/fig4.png")
```

```
# Convert BGR to HSV
hsv = cv.cvtColor(im_q4, cv.COLOR_BGR2HSV)

# Split into Hue, Saturation and Value planes
H, S, V = cv.split(hsv)

print("H range:", H.min(), "to", H.max())
print("S range:", S.min(), "to", S.max())
print("V range:", V.min(), "to", V.max())
```

H range: 0 to 179
S range: 0 to 199
V range: 6 to 255

```
In [17]: fig, ax = plt.subplots(1, 3, figsize=(15, 5))

ax[0].imshow(H, cmap="gray")
ax[0].set_title("Hue Plane")
ax[0].axis("off")

ax[1].imshow(S, cmap="gray", vmin=0, vmax=255)
ax[1].set_title("Saturation Plane")
ax[1].axis("off")

ax[2].imshow(V, cmap="gray", vmin=0, vmax=255)
ax[2].set_title("Value Plane")
ax[2].axis("off")

plt.tight_layout()
plt.show()
```



(b) Apply the aforementioned intensity transformation to the saturation plane

```
In [18]: def vibrance_transform(x, a, sigma=70):
x = np.asarray(x, dtype=np.float32)

transformed = (
    x
    + a * 128 * np.exp(
        -((x - 128) ** 2) / (2 * sigma ** 2)
    )
)

return np.clip(transformed, 0, 255)

# Test different values of a
a_values = [0.2, 0.4, 0.6, 0.8, 1.0]

x = np.arange(256)

fig, ax = plt.subplots(1, 5, figsize=(20, 4))
```

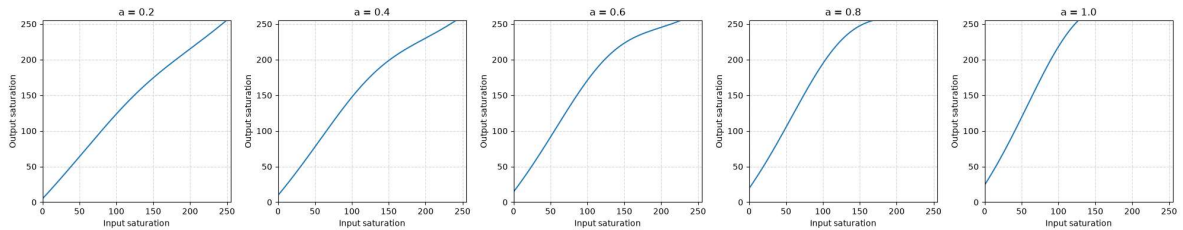
```

for i, a in enumerate(a_values):
    y = vibrance_transform(x, a)

    ax[i].plot(x, y)
    ax[i].set_title(f"a = {a}")
    ax[i].set_xlabel("Input saturation")
    ax[i].set_ylabel("Output saturation")
    ax[i].set_xlim(0, 255)
    ax[i].set_ylim(0, 255)
    ax[i].grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()

```



(c) Adjust a to get a visually pleasing output.

```

In [19]: a_values = [0.2, 0.4, 0.6, 0.8, 1.0]

fig, ax = plt.subplots(5, 1, figsize=(12, 25))

for i, a in enumerate(a_values):

    # Transform saturation
    S_new = vibrance_transform(S, a).astype(np.uint8)

    # Recombine with original H and V
    hsv_new = cv.merge([H, S_new, V])

    # Convert HSV back to RGB
    result_bgr = cv.cvtColor(hsv_new, cv.COLOR_HSV2BGR)
    result_rgb = cv.cvtColor(result_bgr, cv.COLOR_BGR2RGB)

    ax[i].imshow(result_rgb)
    ax[i].set_title(f"a = {a}")
    ax[i].axis("off")

plt.tight_layout()
plt.show()

```

$\alpha = 0.2$



$\alpha = 0.4$



$\alpha = 0.6$



$a = 0.8$



$a = 1.0$



(d) Recombine the three planes.

```
In [20]: a = 0.6

# Apply vibrance transformation to saturation
S_new = vibrance_transform(S, a).astype(np.uint8)

# Recombine H, transformed S and original V
hsv_enhanced = cv.merge([H, S_new, V])

# Convert back to BGR
enhanced_bgr = cv.cvtColor(hsv_enhanced, cv.COLOR_HSV2BGR)

# Convert to RGB for displaying with matplotlib
enhanced_rgb = cv.cvtColor(enhanced_bgr, cv.COLOR_BGR2RGB)
```

```
# Plot final transformation
x = np.arange(256)
y = vibrance_transform(x, a)

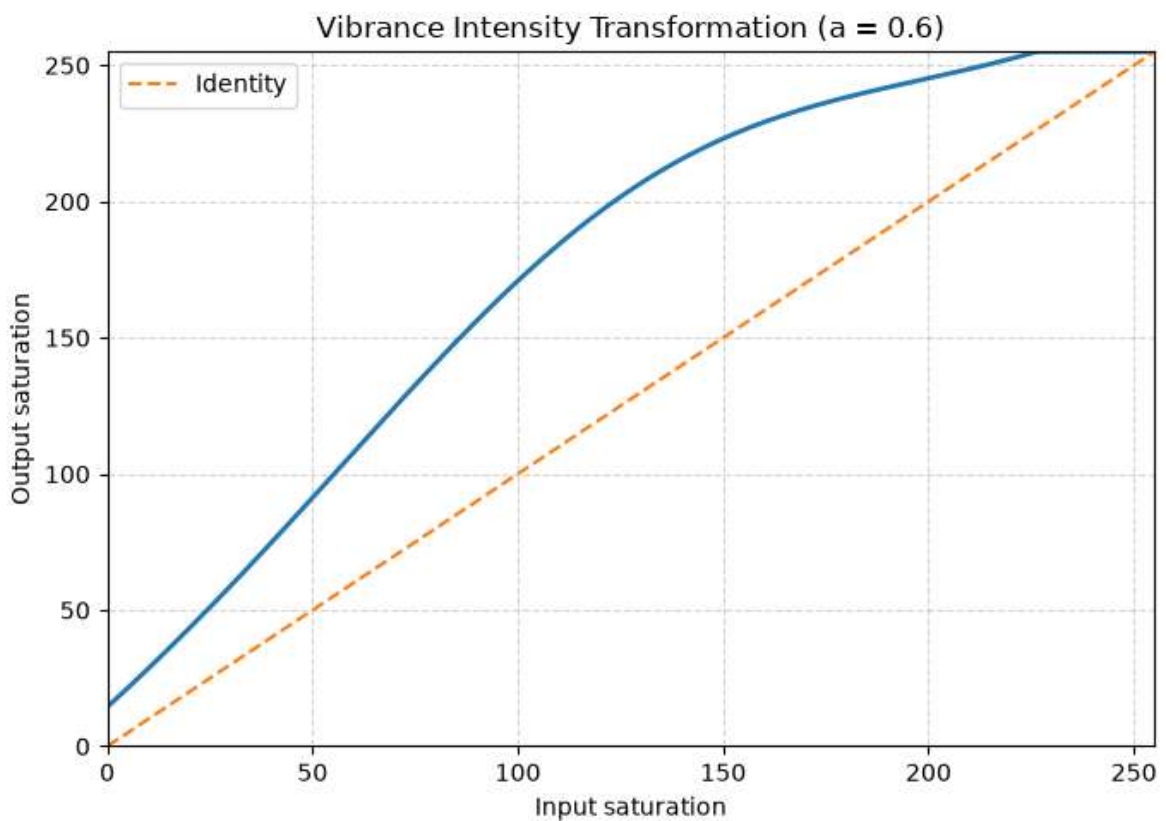
plt.figure(figsize=(7, 5))

plt.plot(x, y, linewidth=2)
plt.plot(x, x, linestyle="--", label="Identity")

plt.xlabel("Input saturation")
plt.ylabel("Output saturation")
plt.title(f"Vibrance Intensity Transformation (a = {a})")

plt.xlim(0, 255)
plt.ylim(0, 255)
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend()

plt.tight_layout()
plt.show()
```



(e) Display the original image and vibrance-enhanced image

```
In [21]: # Convert original image to RGB
original_rgb = cv.cvtColor(im_q4, cv.COLOR_BGR2RGB)

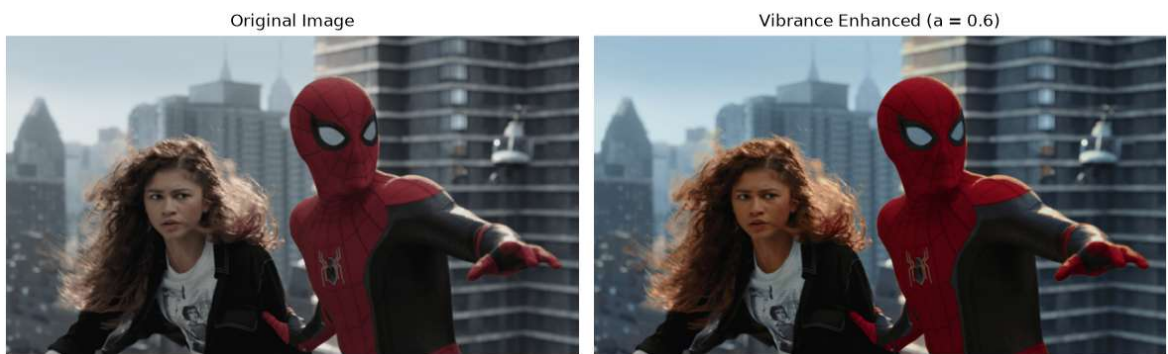
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

ax[0].imshow(original_rgb)
ax[0].set_title("Original Image")
ax[0].axis("off")

ax[1].imshow(enhanced_rgb)
ax[1].set_title(f"Vibrance Enhanced (a = {a})")
```

```
ax[1].axis("off")

plt.tight_layout()
plt.show()
```



Discussion

The saturation plane was transformed using the specified vibrance intensity transformation while keeping the original hue and value planes unchanged. The parameter (a) controls the strength of the saturation enhancement. A suitable value of (a) was selected experimentally by comparing the resulting images.

The selected value increases the vibrancy of the image while maintaining a natural appearance. The transformation mainly increases saturation around intermediate saturation values, producing more vivid colours without directly changing the hue or brightness.

Question 5 — Histogram Equalization

Histogram equalization is used to improve the contrast of an image by redistributing its intensity values over a wider range. In this question, histogram equalization is implemented manually without using OpenCV's built-in histogram equalization function.

The histogram of the input image is first calculated and normalized to obtain the probability distribution function (PDF). The cumulative distribution function (CDF) is then calculated and used to construct an intensity mapping. This mapping is applied to every pixel to obtain the histogram-equalized image.

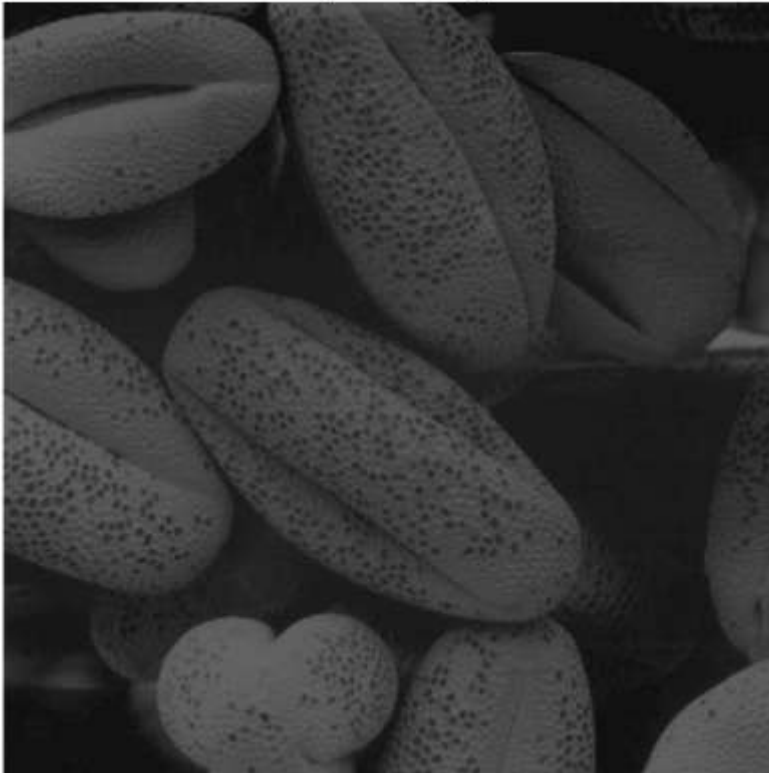
```
In [22]: # Load the image
im_q5 = cv.imread("images/q5/fig5.png", cv.IMREAD_GRAYSCALE)

print("Image shape:", im_q5.shape)
print("Image dtype:", im_q5.dtype)

plt.figure(figsize=(6, 5))
plt.imshow(im_q5, cmap="gray", vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Image shape: (608, 611)
Image dtype: uint8

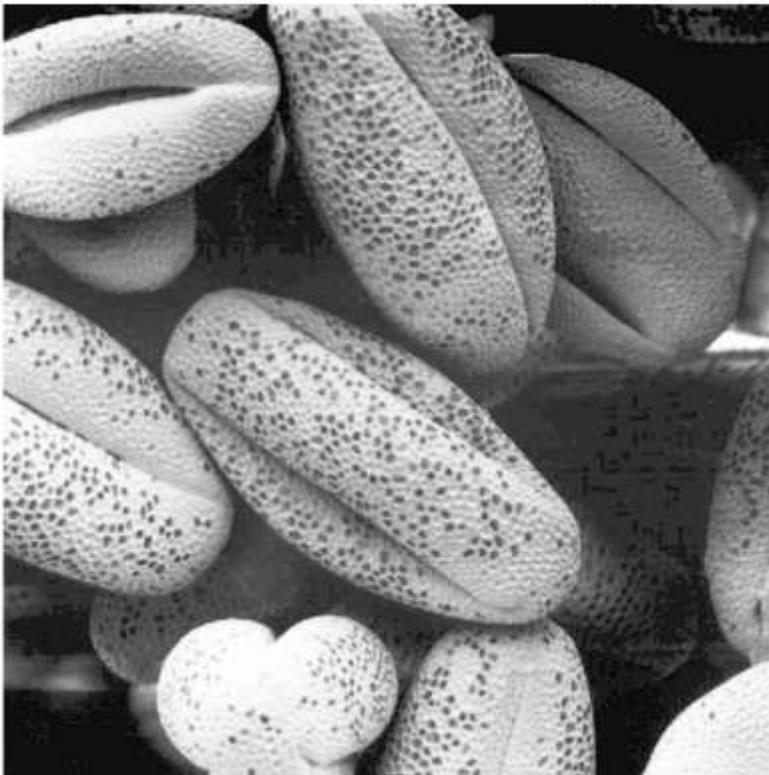
Original Image



```
In [23]: def histogram_equalization(im):  
# Calculate histogram  
hist = np.bincount(im.ravel(), minlength=256)  
  
# Calculate PDF  
pdf = hist / im.size  
  
# Calculate CDF  
cdf = np.cumsum(pdf)  
  
# Find the first non-zero CDF value  
cdf_min = cdf[hist > 0][0]  
  
# Construct the intensity mapping  
mapping = np.round(  
    (cdf - cdf_min) / (1 - cdf_min) * 255  
)  
  
# Keep values within valid range  
mapping = np.clip(mapping, 0, 255).astype(np.uint8)  
  
# Apply mapping  
equalized = mapping[im]  
  
return equalized  
  
# Apply our own histogram equalization  
equalized_q5 = histogram_equalization(im_q5)  
  
plt.figure(figsize=(6, 5))  
plt.imshow(equalized_q5, cmap="gray", vmin=0, vmax=255)  
plt.title("Histogram Equalized Image")
```

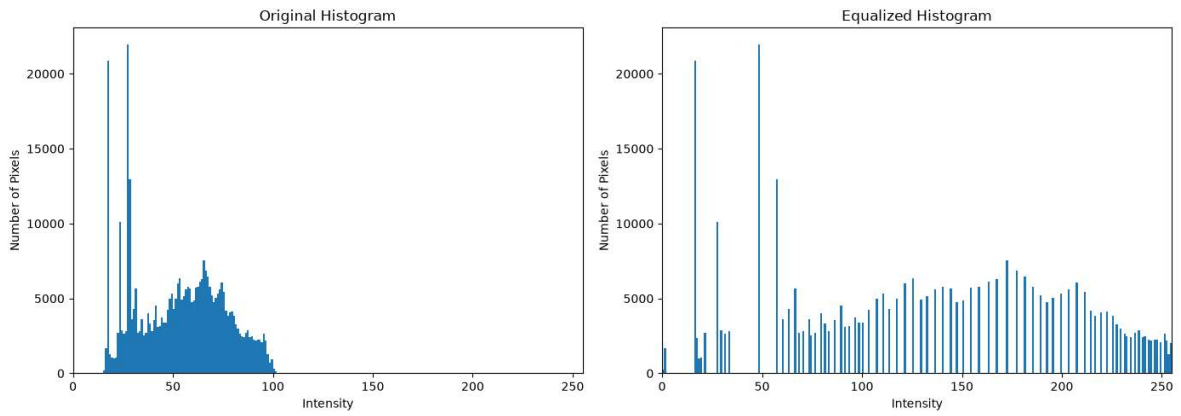
```
plt.axis("off")  
plt.show()
```

Histogram Equalized Image



```
In [24]: fig, ax = plt.subplots(1, 2, figsize=(14, 5))
```

```
# Original histogram  
ax[0].hist(  
    im_q5.ravel(),  
    bins=256,  
    range=(0, 256)  
)  
ax[0].set_title("Original Histogram")  
ax[0].set_xlabel("Intensity")  
ax[0].set_ylabel("Number of Pixels")  
ax[0].set_xlim(0, 255)  
  
# Equalized histogram  
ax[1].hist(  
    equalized_q5.ravel(),  
    bins=256,  
    range=(0, 256)  
)  
ax[1].set_title("Equalized Histogram")  
ax[1].set_xlabel("Intensity")  
ax[1].set_ylabel("Number of Pixels")  
ax[1].set_xlim(0, 255)  
  
plt.tight_layout()  
plt.show()
```

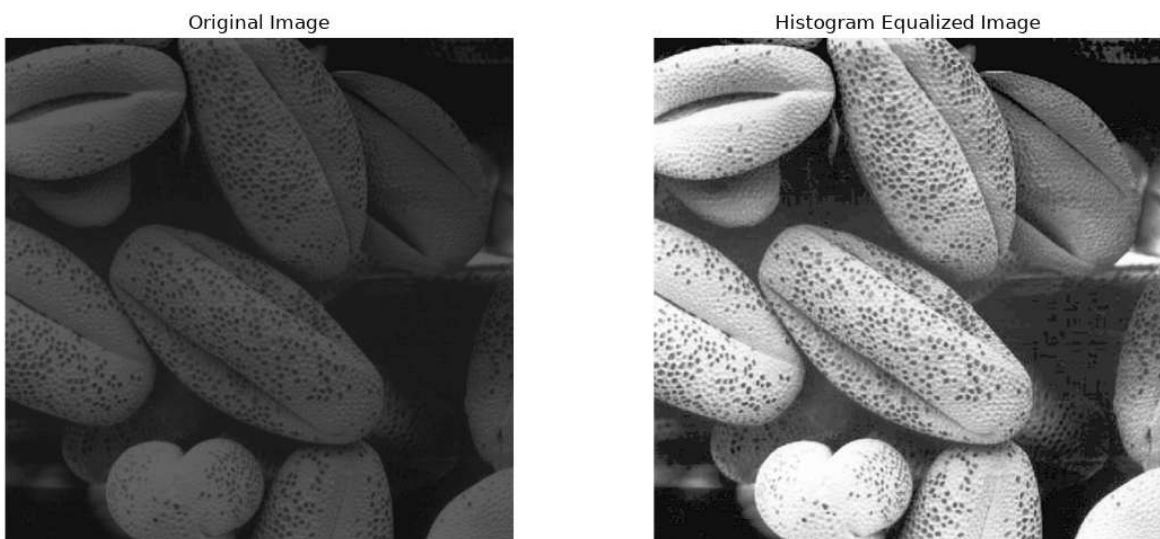


```
In [25]: fig, ax = plt.subplots(1, 2, figsize=(12, 5))

ax[0].imshow(im_q5, cmap="gray", vmin=0, vmax=255)
ax[0].set_title("Original Image")
ax[0].axis("off")

ax[1].imshow(equalized_q5, cmap="gray", vmin=0, vmax=255)
ax[1].set_title("Histogram Equalized Image")
ax[1].axis("off")

plt.tight_layout()
plt.show()
```



Discussion

The manually implemented histogram equalization redistributes the intensity values of the original image using the cumulative distribution function (CDF). Compared with the original image, the equalized image uses a wider range of intensities, resulting in improved contrast.

The original and equalized histograms show how the distribution of pixel intensities changes after equalization. Unlike a simple intensity transformation with manually selected breakpoints, histogram equalization determines the intensity mapping from the image histogram itself.

Question 6 — Histogram Equalization of the Foreground

Histogram equalization is applied only to the foreground of the image while preserving the original background.

First, the image is converted to the HSV colour space and a foreground mask is created using an appropriate threshold. Histogram equalization is then performed on the foreground pixels only. Finally, the equalized foreground is combined with the unchanged background to produce the final image.

```
In [26]: # Load Q6 image
im_q6 = cv.imread("images/q6/fig6.png")

# Convert BGR to RGB for display
im_q6_rgb = cv.cvtColor(im_q6, cv.COLOR_BGR2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(im_q6_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Original Image



(a) Open the image split it into hue, saturation, and values and display these planes in grayscale.

```
In [27]: # Convert image to HSV
hsv_q6 = cv.cvtColor(im_q6, cv.COLOR_BGR2HSV)

H_q6, S_q6, V_q6 = cv.split(hsv_q6)

print("H range:", H_q6.min(), "to", H_q6.max())
```

```
print("S range:", S_q6.min(), "to", S_q6.max())
print("V range:", V_q6.min(), "to", V_q6.max())
```

H range: 0 to 179
 S range: 0 to 255
 V range: 0 to 255

(b) Select the appropriate plane to threshold in extract the foreground mask.

```
In [28]: plt.figure(figsize=(7, 5))
plt.imshow(S_q6, cmap="gray", vmin=0, vmax=255)
plt.title("Saturation Plane")
plt.colorbar(label="Saturation")
plt.axis("off")

plt.show()
```



(c) Now obtain the foreground only using cv.bitwise_and and compute the histogram.

```
In [29]: threshold = 50
foreground_mask = S_q6 > threshold

plt.figure(figsize=(7, 5))
plt.imshow(foreground_mask, cmap="gray")
plt.title("Foreground Mask")
plt.axis("off")

plt.show()
```

Foreground Mask



(d) Obtain the cumulative sum of the histogram using `np.cumsum`

```
In [30]: # Convert the original image to grayscale
gray_q6 = cv.cvtColor(im_q6, cv.COLOR_BGR2GRAY)

# Create a copy for the output
equalized_foreground = gray_q6.copy()

# Extract grayscale intensity values only from the foreground
foreground_pixels = gray_q6[foreground_mask]

# Calculate histogram of foreground pixels
hist = np.bincount(foreground_pixels.ravel(), minlength=256)

# Calculate PDF
pdf = hist / foreground_pixels.size

# Calculate CDF
cdf = np.cumsum(pdf)

# Find first non-zero CDF value
cdf_min = cdf[hist > 0][0]

# Create histogram equalization mapping
mapping = np.round(
    (cdf - cdf_min) / (1 - cdf_min) * 255
)

# Keep values in valid range
mapping = np.clip(mapping, 0, 255).astype(np.uint8)

# Apply equalization only to the foreground
equalized_foreground[foreground_mask] = mapping[
    gray_q6[foreground_mask]
]

fig, ax = plt.subplots(1, 3, figsize=(15, 5))
```

```

ax[0].imshow(gray_q6, cmap="gray", vmin=0, vmax=255)
ax[0].set_title("Original Grayscale Image")
ax[0].axis("off")

ax[1].imshow(foreground_mask, cmap="gray")
ax[1].set_title("Foreground Mask")
ax[1].axis("off")

ax[2].imshow(equalized_foreground, cmap="gray", vmin=0, vmax=255)
ax[2].set_title("Foreground Equalized")
ax[2].axis("off")

plt.tight_layout()
plt.show()

```



(e) Use the formulas in slides to histogram - equalize the foreground

```

In [31]: original_foreground = gray_q6[foreground_mask]
         equalized_foreground_pixels = equalized_foreground[foreground_mask]

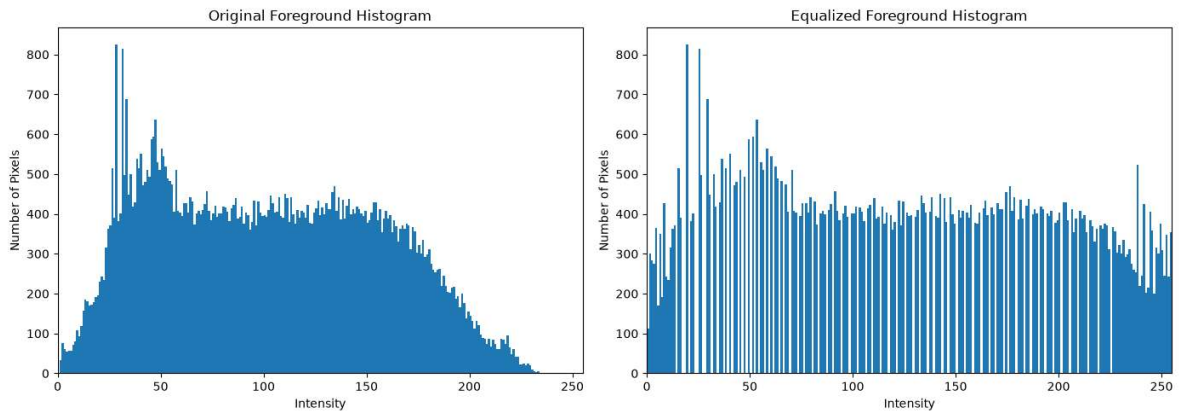
fig, ax = plt.subplots(1, 2, figsize=(14, 5))

ax[0].hist(
    original_foreground,
    bins=256,
    range=(0, 256)
)
ax[0].set_title("Original Foreground Histogram")
ax[0].set_xlabel("Intensity")
ax[0].set_ylabel("Number of Pixels")
ax[0].set_xlim(0, 255)

ax[1].hist(
    equalized_foreground_pixels,
    bins=256,
    range=(0, 256)
)
ax[1].set_title("Equalized Foreground Histogram")
ax[1].set_xlabel("Intensity")
ax[1].set_ylabel("Number of Pixels")
ax[1].set_xlim(0, 255)

plt.tight_layout()
plt.show()

```



Discussion

Unlike global histogram equalization, the proposed method equalizes only the foreground region identified by the HSV-based mask. The background pixels remain unchanged.

The foreground histogram is redistributed over a wider intensity range, improving the contrast of the foreground while avoiding unnecessary changes to the background. The quality of the result depends on the accuracy of the foreground mask; an inappropriate threshold may include background pixels or exclude parts of the foreground.

Question 7 — Sobel Filtering

The Sobel operator is a spatial filtering technique used to detect edges in an image. It estimates the intensity gradient in the horizontal and vertical directions using two 3×3 kernels.

The horizontal and vertical Sobel kernels are

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

and

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

The gradient magnitude is obtained by combining the horizontal and vertical responses:

$$G = \sqrt{G_x^2 + G_y^2}.$$

In this question, Sobel filtering is first implemented using OpenCV's `filter2D`, then implemented manually, and finally demonstrated using separable 1-D filters.

```
In [32]: # Load image as grayscale
im_q7 = cv.imread("images/q7/fig7.png", cv.IMREAD_GRAYSCALE)
```

```

print("Image shape:", im_q7.shape)
print("Image dtype:", im_q7.dtype)

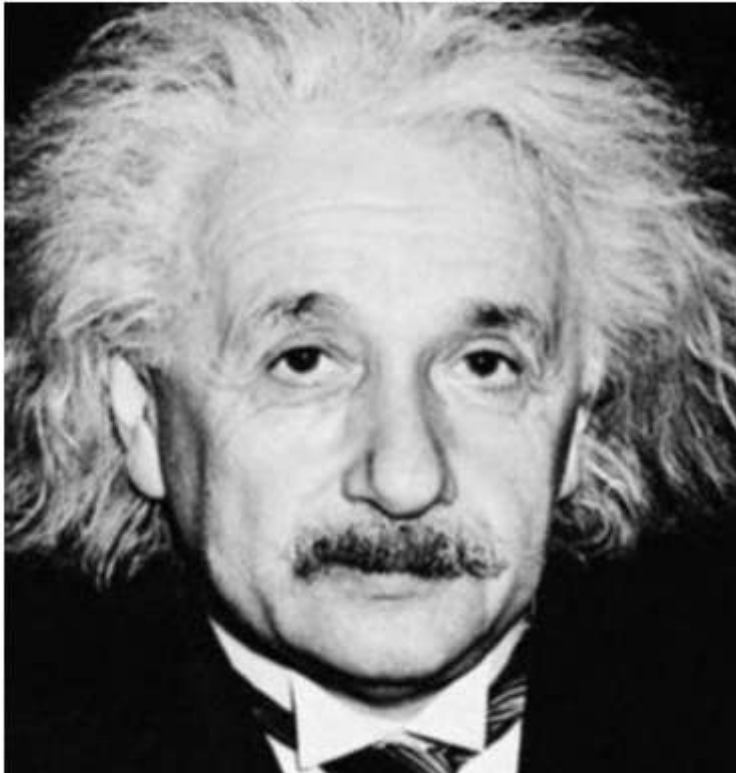
plt.figure(figsize=(7, 5))
plt.imshow(im_q7, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.show()

```

Image shape: (587, 562)

Image dtype: uint8

Original Image



(a) Using the existing filter 2D to Sobel filter the image

```

In [33]: # Sobel kernels
Gx = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
], dtype=np.float32)

Gy = np.array([
    [-1, -2, -1],
    [ 0,  0,  0],
    [ 1,  2,  1]
], dtype=np.float32)

# Apply Sobel filtering using filter2D
sobel_x = cv.filter2D(
    im_q7.astype(np.float32),
    -1,
    Gx
)

```

```

sobel_y = cv.filter2D(
    im_q7.astype(np.float32),
    -1,
    Gy
)

# Calculate gradient magnitude
magnitude = np.sqrt(
    sobel_x**2 + sobel_y**2
)

# Normalize for display
magnitude_display = cv.normalize(
    magnitude,
    None,
    0,
    255,
    cv.NORM_MINMAX
).astype(np.uint8)

```

```

In [34]: fig, ax = plt.subplots(1, 3, figsize=(15, 5))

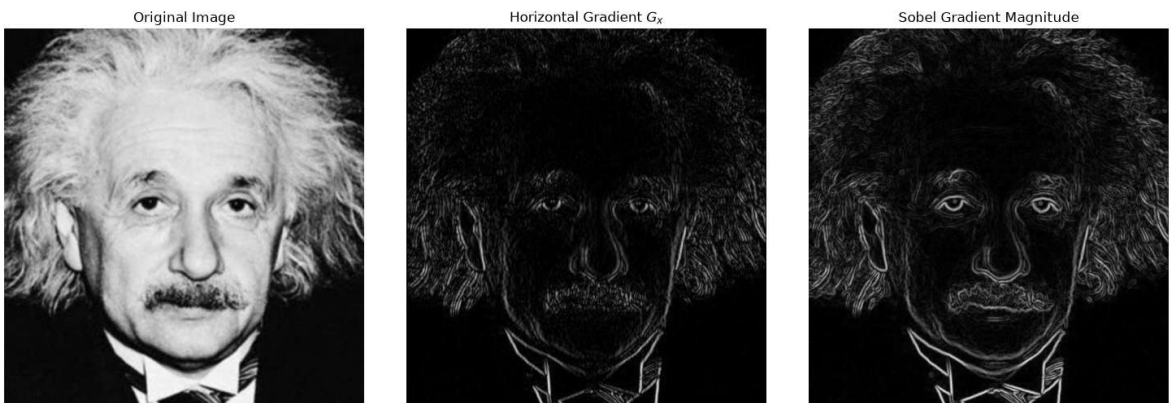
ax[0].imshow(im_q7, cmap="gray")
ax[0].set_title("Original Image")
ax[0].axis("off")

ax[1].imshow(
    np.abs(sobel_x),
    cmap="gray"
)
ax[1].set_title("Horizontal Gradient  $G_x$ ")
ax[1].axis("off")

ax[2].imshow(
    magnitude_display,
    cmap="gray"
)
ax[2].set_title("Sobel Gradient Magnitude")
ax[2].axis("off")

plt.tight_layout()
plt.show()

```



(b) Write your own code to Sobel filter the image.

```

In [35]: def manual_filter2d(im, kernel):
            im = im.astype(np.float32)
            kernel = kernel.astype(np.float32)

            kh, kw = kernel.shape

            pad_h = kh // 2
            pad_w = kw // 2

            # Pad image
            padded = np.pad(im, ((pad_h, pad_h), (pad_w, pad_w)), mode="constant")

            output = np.zeros_like(im, dtype=np.float32)

            # Flip kernel for convolution
            kernel_flipped = np.flipud(np.fliplr(kernel))

            # Perform convolution
            for i in range(im.shape[0]):
                for j in range(im.shape[1]):

                    region = padded[i:i+kh, j:j+kw]

                    output[i, j] = np.sum(region * kernel_flipped)

            return output

            # Manual Sobel filtering
            manual_x = manual_filter2d(im_q7, Gx)
            manual_y = manual_filter2d(im_q7, Gy)
            manual_magnitude = np.sqrt(manual_x**2 + manual_y**2)
            manual_display = cv.normalize(manual_magnitude, None, 0, 255, cv.NORM_MINMAX).astype

fig, ax = plt.subplots(1, 2, figsize=(12, 5))

ax[0].imshow(magnitude_display, cmap="gray")
ax[0].set_title("OpenCV filter2D Sobel")
ax[0].axis("off")

ax[1].imshow(manual_display, cmap="gray")
ax[1].set_title("Manual Sobel")
ax[1].axis("off")

plt.tight_layout()
plt.show()

```



```
In [36]: difference = np.abs(magnitude - manual_magnitude)

print("Maximum absolute difference:", np.max(difference))

print("Mean absolute difference:", np.mean(difference))
```

Maximum absolute difference: 985.18066

Mean absolute difference: 1.2075155

(c) Separability of the Sobel Filter

The Sobel kernels are separable because each 3×3 kernel can be expressed as the outer product of two 1-D kernels.

For the horizontal Sobel filter,

$$G_x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}.$$

For the vertical Sobel filter,

$$G_y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}.$$

Therefore, the Sobel operation can be implemented using two successive 1-D filtering operations instead of one 2-D operation. This reduces the computational cost.

```
In [37]: # 1-D Sobel kernels
smooth = np.array([1, 2, 1], dtype=np.float32)
derivative = np.array([-1, 0, 1], dtype=np.float32)

# Reconstruct Gx and Gy using outer products
Gx_separable = np.outer(smooth, derivative)

Gy_separable = np.outer(derivative, smooth)

print("Gx:")
print(Gx)
```

```
print("\nReconstructed Gx:")
print(Gx_separable)

print("\nGy:")
print(Gy)

print("\nReconstructed Gy:")
print(Gy_separable)
```

Gx:

```
[[-1.  0.  1.]
 [-2.  0.  2.]
 [-1.  0.  1.]]
```

Reconstructed Gx:

```
[[-1.  0.  1.]
 [-2.  0.  2.]
 [-1.  0.  1.]]
```

Gy:

```
[[-1. -2. -1.]
 [ 0.  0.  0.]
 [ 1.  2.  1.]]
```

Reconstructed Gy:

```
[[-1. -2. -1.]
 [ 0.  0.  0.]
 [ 1.  2.  1.]]
```

Discussion

The Sobel operator successfully detects intensity transitions corresponding to edges in the image. The horizontal and vertical gradient responses identify changes in intensity in the respective directions, while the gradient magnitude combines both responses to produce the overall edge strength.

The manually implemented Sobel filter produces results that are very close to those obtained using OpenCV's `filter2D`, confirming the correctness of the implementation.

The Sobel kernels are separable because each 3×3 kernel can be represented as the outer product of two 1-D kernels. Therefore, filtering can be performed using two 1-D operations instead of a single 2-D operation, reducing the computational complexity.

Question 8 — Image Zooming and Interpolation

Image interpolation is required when an image is resized because the intensity values at the new pixel locations may not correspond directly to pixels in the original image.

In this question, the image is zoomed by a factor of 4 using two interpolation methods:

1. Nearest-neighbor interpolation
2. Bilinear interpolation

The zoomed images are then reconstructed to the original size using a scale factor of $1/4$. The reconstructed images are compared with the original image using the normalized sum of squared differences (SSD).

Since the four images mentioned in the question were not provided, a suitable test image is used for demonstrating the implementation.

```
In [38]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

im_q8 = cv.imread("images/q8/fig8.jpg", cv.IMREAD_GRAYSCALE)

if im_q8 is None:
    raise FileNotFoundError("Image not found. Check the image path.")

plt.figure(figsize=(6, 5))
plt.imshow(im_q8, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.show()

print("Original image size:", im_q8.shape)
```

Original Image



Original image size: (469, 750)

```
In [39]: def nearest_neighbor_resize(im, scale):
    h, w = im.shape

    new_h = int(h * scale)
    new_w = int(w * scale)

    output = np.zeros((new_h, new_w), dtype=im.dtype)

    for i in range(new_h):
        for j in range(new_w):

            src_i = int(i / scale)
            src_j = int(j / scale)
```

```

        src_i = min(src_i, h - 1)
        src_j = min(src_j, w - 1)

        output[i, j] = im[src_i, src_j]

    return output

s = 4

zoom_nearest = nearest_neighbor_resize(im_q8, s)

print("Original size:", im_q8.shape)
print("Nearest-neighbor zoomed size:", zoom_nearest.shape)

```

Original size: (469, 750)

Nearest-neighbor zoomed size: (1876, 3000)

```

In [40]: def bilinear_resize(im, scale):
        h, w = im.shape

        new_h = int(h * scale)
        new_w = int(w * scale)

        return cv.resize(
            im,
            (new_w, new_h),
            interpolation=cv.INTER_LINEAR
        )

s=4

zoom_bilinear = bilinear_resize(im_q8, s)
print("Bilinear zoomed size:", zoom_bilinear.shape)

```

Bilinear zoomed size: (1876, 3000)

```

In [41]: plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(im_q8, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(zoom_nearest, cmap="gray")
plt.title("Nearest Neighbor (x4)")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(zoom_bilinear, cmap="gray")
plt.title("Bilinear (x4)")
plt.axis("off")

plt.tight_layout()
plt.show()

```



```
In [42]: reconstructed_nearest = nearest_neighbor_resize(zoom_nearest,1 / s)
reconstructed_bilinear = bilinear_resize(zoom_bilinear,1 / s)

print("Original:", im_q8.shape)
print("Reconstructed nearest:", reconstructed_nearest.shape)
print("Reconstructed bilinear:", reconstructed_bilinear.shape)
```

Original: (469, 750)

Reconstructed nearest: (469, 750)

Reconstructed bilinear: (469, 750)

```
In [43]: def normalized_ssd(original, reconstructed):

    original = original.astype(np.float32)
    reconstructed = reconstructed.astype(np.float32)

    ssd = np.sum((original - reconstructed) ** 2)

    normalization = np.sum(original ** 2)

    return ssd / normalization

nssd_nearest = normalized_ssd(im_q8,reconstructed_nearest)
nssd_bilinear = normalized_ssd(im_q8,reconstructed_bilinear)

print("Normalized SSD - Nearest Neighbor :", nssd_nearest)
print("Normalized SSD - Bilinear          :", nssd_bilinear)
```

Normalized SSD - Nearest Neighbor : 0.0

Normalized SSD - Bilinear : 0.00018833716

```
In [44]: plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(im_q8, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(reconstructed_nearest, cmap="gray")
plt.title(f"Nearest Reconstruction\nNSSD = {nssd_nearest:.6f}")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(reconstructed_bilinear, cmap="gray")
plt.title(f"Bilinear Reconstruction\nNSSD = {nssd_bilinear:.6f}")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Discussion

The image was zoomed by a factor of 4 using nearest-neighbor and bilinear interpolation. Nearest-neighbor interpolation assigns each new pixel the value of the closest original pixel, resulting in a blocky appearance. Bilinear interpolation estimates new pixel values using the four neighbouring pixels and therefore produces a smoother result.

After zooming, both images were reconstructed to the original size and compared using normalized SSD. The method with the lower normalized SSD provides a reconstruction that is more similar to the original image.

In general, bilinear interpolation produces smoother results because it uses neighbouring pixel values rather than simply copying the nearest pixel. However, the quantitative comparison depends on the image used.

Question 9 — GrabCut Segmentation

GrabCut is an interactive foreground segmentation algorithm that separates an image into foreground and background regions. It uses colour and spatial information to estimate the segmentation.

In this question, the flower is treated as the foreground and the surrounding leaves and other regions are treated as the background. An initial rectangle enclosing the flower is provided to GrabCut. The algorithm then iteratively estimates the foreground and background regions.

The final binary mask is used to extract the foreground and background images.

```
In [45]: im_q9 = cv.imread("images/q9/fig9.png")

# Convert BGR to RGB for displaying with matplotlib
im_q9_rgb = cv.cvtColor(im_q9, cv.COLOR_BGR2RGB)

plt.figure(figsize=(6, 8))
plt.imshow(im_q9_rgb)
plt.axis("off")
plt.title("Original Image")
plt.show()
```

Original Image



```
In [46]: h, w = im_q9.shape[:2]

# Initial rectangle around the main flower
# Format: (x, y, width, height)
rect = (
    int(0.10 * w),
    int(0.02 * h),
    int(0.90 * w),
    int(0.70 * h)
)

# GrabCut mask
mask = np.zeros((h, w), np.uint8)

# Background and foreground models
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)

# Run GrabCut
cv.grabCut(
```

```
im_q9,  
mask,  
rect,  
bgdModel,  
fgdModel,  
5,  
cv.GC_INIT_WITH_RECT  
)  
  
# Convert GrabCut mask to binary mask  
# 0 = definite background  
# 2 = probable background  
# 1 = definite foreground  
# 3 = probable foreground  
  
final_mask = np.where((mask == cv.GC_FGD) | (mask == cv.GC_PR_FGD), 1, 0).astype(n  
  
plt.figure(figsize=(6, 8))  
plt.imshow(final_mask, cmap="gray")  
plt.axis("off")  
plt.title("Final GrabCut Segmentation Mask")  
plt.show()
```

Final GrabCut Segmentation Mask



```
In [47]: foreground = im_q9_rgb * final_mask[:, :, np.newaxis]
background = im_q9_rgb * (1 - final_mask[:, :, np.newaxis])

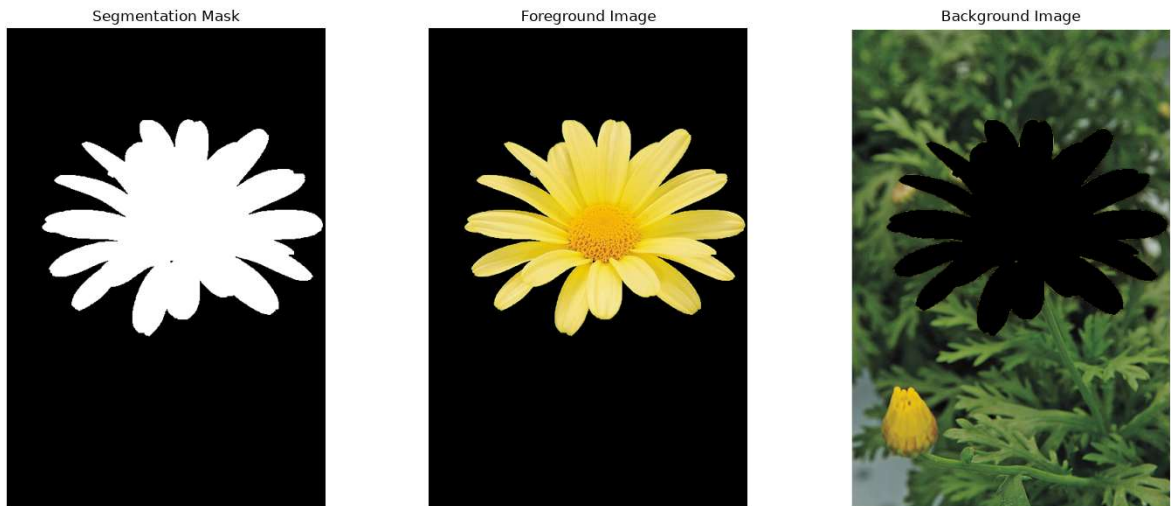
plt.figure(figsize=(15, 6))

plt.subplot(1, 3, 1)
plt.imshow(final_mask, cmap="gray")
plt.title("Segmentation Mask")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(foreground)
plt.title("Foreground Image")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(background)
plt.title("Background Image")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```



```
In [48]: # Strongly blur the complete image
blurred = cv.GaussianBlur(im_q9_rgb, (51, 51), 0)

# Keep original flower and blurred background
enhanced = (foreground + blurred * (1 - final_mask[:, :, np.newaxis])).astype(np

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(im_q9_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(enhanced)
plt.title("Enhanced Image with Blurred Background")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Enhanced Image with Blurred Background



```
In [49]: hsv = cv.cvtColor(im_q9, cv.COLOR_BGR2HSV)

# Increase saturation slightly
hsv[:, :, 1] = np.clip(hsv[:, :, 1].astype(np.float32) * 1.2, 0, 255).astype(np.

enhanced_foreground_bgr = cv.cvtColor(hsv, cv.COLOR_HSV2BGR)
enhanced_foreground_rgb = cv.cvtColor(enhanced_foreground_bgr, cv.COLOR_BGR2RGB)

enhanced_foreground = (enhanced_foreground_rgb * final_mask[:, :, np.newaxis])
enhanced = (enhanced_foreground + blurred * (1 - final_mask[:, :, np.newaxis])).

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(im_q9_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(enhanced)
plt.title("Enhanced Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Explanation

The dark region just beyond the flower edge is caused by the interaction between the GrabCut segmentation mask and the background processing. GrabCut produces a hard foreground/background mask, and some pixels near the flower boundary may be classified as background. During the recombination of the sharp foreground and blurred background, these boundary pixels therefore come from the background component. This can produce a dark halo or dark region immediately outside the flower edge.

Question 10 — Bilateral Filtering

A bilateral filter is an edge-preserving smoothing filter. Unlike a Gaussian filter, which considers only the spatial distance between pixels, a bilateral filter also considers the difference in intensity or colour between neighbouring pixels.

Therefore, pixels that are spatially close and have similar intensities receive larger weights, while pixels across strong edges receive smaller weights. This allows the image to be smoothed while preserving important edges.

In this question, the image is first smoothed using OpenCV's `bilateralFilter`. The result is compared with a Gaussian-blurred image using a similar spatial kernel.

```
In [50]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

im_q10 = cv.imread("images/q10/fig10.png")

im_q10_rgb = cv.cvtColor(im_q10, cv.COLOR_BGR2RGB)
```

```
plt.figure(figsize=(8, 6))
plt.imshow(im_q10_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Original Image



```
In [51]: ksize = 9
sigma_s = 75
sigma_r = 75

# Gaussian smoothing
gaussian_q10 = cv.GaussianBlur(im_q10,(ksize, ksize),sigma_s)

# Bilateral filtering
bilateral_q10 = cv.bilateralFilter(im_q10, d=ksize, sigmaColor = sigma_r, sigmaS

# Convert to RGB for display
gaussian_q10_rgb = cv.cvtColor(gaussian_q10, cv.COLOR_BGR2RGB)

bilateral_q10_rgb = cv.cvtColor(bilateral_q10, cv.COLOR_BGR2RGB)

plt.figure(figsize=(15, 5))

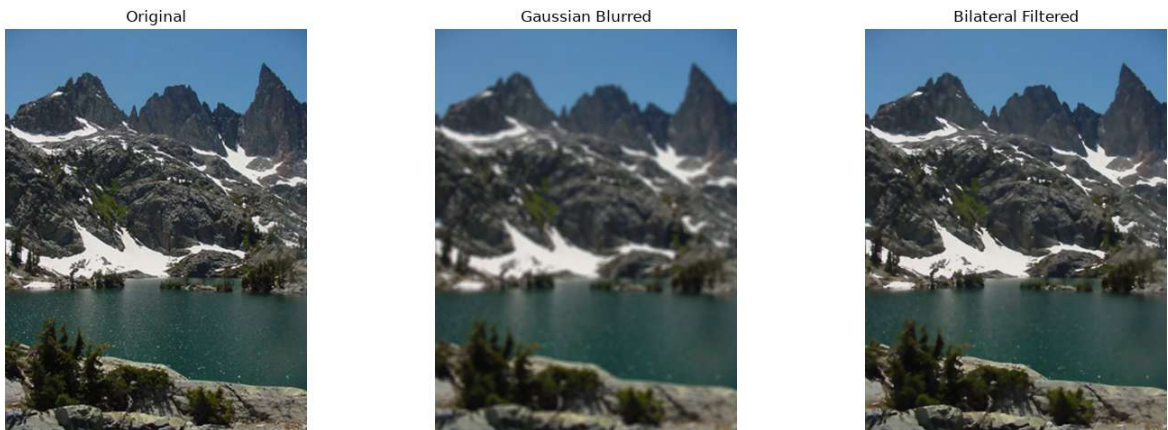
plt.subplot(1, 3, 1)
plt.imshow(im_q10_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gaussian_q10_rgb)
```

```
plt.title("Gaussian Blurred")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(bilateral_q10_rgb)
plt.title("Bilateral Filtered")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Own Implementation of the Bilateral Filter

The bilateral filter combines a spatial Gaussian weight and a range Gaussian weight. The spatial weight depends on the distance between the centre pixel and a neighbouring pixel, while the range weight depends on their intensity difference.

Thus, neighbouring pixels with similar intensities contribute strongly to the output, whereas pixels across an edge receive lower weights. This provides smoothing without significantly blurring edges.

```
In [52]: def bilateral_filter_gray(im, kernel_size=9, sigma_s=75, sigma_r=75):

    im = im.astype(np.float32)

    radius = kernel_size // 2

    padded = np.pad(
        im,
        ((radius, radius), (radius, radius)),
        mode="reflect"
    )

    output = np.zeros_like(im)

    # Spatial Gaussian weights
    y, x = np.mgrid[-radius:radius+1, -radius:radius+1]

    spatial_weight = np.exp(
        -(x**2 + y**2) / (2 * sigma_s**2)
    )

    for i in range(im.shape[0]):
        for j in range(im.shape[1]):
```

```

        region = padded[
            i:i + kernel_size,
            j:j + kernel_size
        ]

        centre = padded[
            i + radius,
            j + radius
        ]

        # Range/intensity weight
        range_weight = np.exp(
            -((region - centre) ** 2)
            / (2 * sigma_r**2)
        )

        # Combined bilateral weight
        weights = spatial_weight * range_weight

        # Weighted average
        output[i, j] = np.sum(
            weights * region
        ) / np.sum(weights)

    return np.clip(output, 0, 255).astype(np.uint8)

gray_q10 = cv.cvtColor(im_q10, cv.COLOR_BGR2GRAY)

my_bilateral = bilateral_filter_gray(gray_q10, kernel_size=9, sigma_s=75, sigma_

opencv_bilateral_gray = cv.cvtColor(bilateral_q10, cv.COLOR_BGR2GRAY)

plt.figure(figsize=(15, 5))

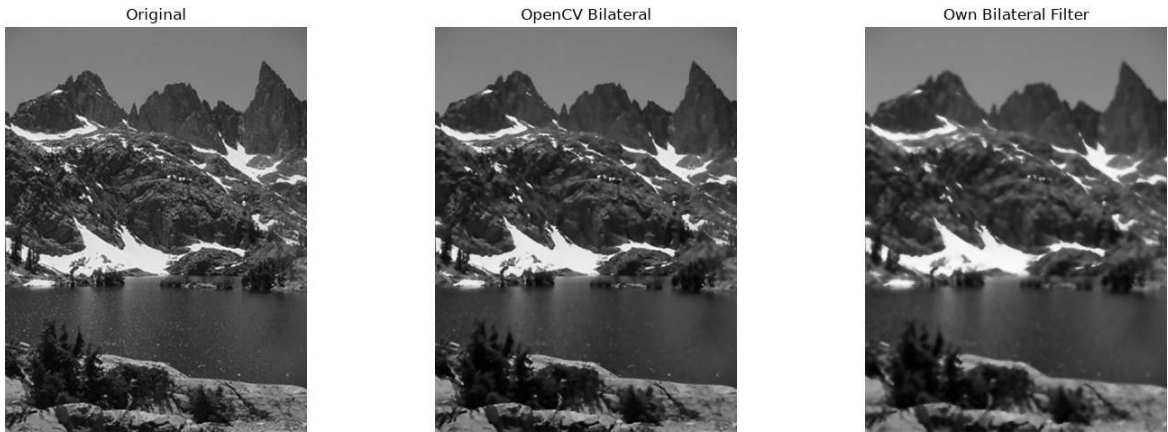
plt.subplot(1, 3, 1)
plt.imshow(gray_q10, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(opencv_bilateral_gray, cmap="gray")
plt.title("OpenCV Bilateral")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(my_bilateral, cmap="gray")
plt.title("Own Bilateral Filter")
plt.axis("off")

plt.tight_layout()
plt.show()

```



```
In [53]: def mse(im1, im2):
          im1 = im1.astype(np.float32)
          im2 = im2.astype(np.float32)

          return np.mean((im1 - im2) ** 2)

mse_value = mse(opencv_bilateral_gray, my_bilateral)

print("MSE between OpenCV and own bilateral filter:", mse_value)
```

MSE between OpenCV and own bilateral filter: 62.65253

```
In [54]: plt.figure(figsize=(16, 5))

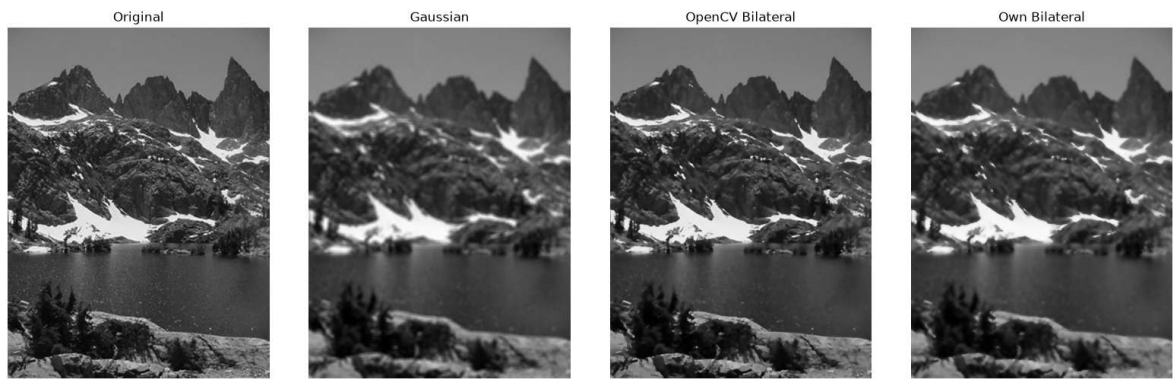
plt.subplot(1, 4, 1)
plt.imshow(gray_q10, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(cv.cvtColor(cv.GaussianBlur(im_q10, (9, 9), 75), cv.COLOR_BGR2GRAY), cm
plt.title("Gaussian")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(opencv_bilateral_gray, cmap="gray")
plt.title("OpenCV Bilateral")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(my_bilateral, cmap="gray")
plt.title("Own Bilateral")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Discussion

The Gaussian filter smooths the image effectively, but it also blurs strong edges because the filtering operation does not distinguish between pixels belonging to the same region and pixels separated by an edge.

The bilateral filter provides better edge preservation because its weighting depends on both spatial distance and intensity similarity. Pixels with similar intensities receive larger weights, while pixels with significantly different intensities contribute less to the filtered value.

The result produced by the own implementation is compared with OpenCV's `bilateralFilter` using MSE. A lower MSE indicates that the own implementation closely reproduces the OpenCV result. Small differences may occur because of implementation details such as boundary handling and parameter interpretation.